

Generación Automática de Mapas Conceptuales con formato Mermaid mediante un Pipeline Híbrido Neuro-Simbólico con Modelos de Lenguaje Pequeños Locales

Aldasoro, Lautaro Gabriel

*Laboratorio de Investigación de Software (LIS) — Universidad Tecnológica
Nacional, Facultad Regional Córdoba*

Nota: versión preliminar del trabajo (en preparación para CoNaIIISI). Documento re-compuesto a partir del texto original en

Google Drive para su publicación en el sitio del LIS.

Abstract

Este trabajo presenta un agente para la generación automática de mapas conceptuales en formato Mermaid a partir de texto libre, utilizando un Modelo de Lenguaje Pequeño (SLM) ejecutado localmente. Es importante destacar que, en el diseño propuesto, el SLM no genera el diagrama en sí. Su función se limita a extraer datos semánticos estructurados (conceptos y relaciones), que luego un componente programático independiente traduce a código Mermaid válido. A diferencia de enfoques previos que dependen de modelos comerciales en la nube para generar la totalidad de la estructura del diagrama, se propone una arquitectura híbrida de cuatro fases que separa la extracción semántica —a cargo del SLM— de la construcción y validación estructural del grafo, a cargo de componentes programáticos. El sistema sigue explícitamente la teoría de mapas conceptuales de Novak y Cañas, implementando sus constructos de Pregunta de Enfoque, Parking Lot y Enlaces Cruzados. La validación de calidad se realiza mediante un evaluador híbrido neuro-simbólico compuesto por diez métricas —nueve deterministas y una semántica basada en el propio SLM— que habilitan un ciclo de autocorrección con reintentos. Sobre un corpus de siete textos de prueba, el sistema alcanzó tasas de aprobación de entre 14.3% y 100% a través de cuatro corridas independientes (media $64.3\% \pm 35.9$ puntos porcentuales), revelando una variabilidad estocástica considerable entre invocaciones del SLM que constituye uno de los principales hallazgos de este trabajo, junto con la identificación de la densidad de interconexión como el cuello de botella más persistente en la calidad estructural de los mapas generados. Los resultados preliminares sugieren que es posible obtener mapas conceptuales estructuralmente sólidos utilizando modelos de

bajo costo computacional ejecutados localmente, aunque una caracterización confiable del sistema requiere un diseño experimental con repetición que esta primera versión del trabajo no incluye aún.

Palabras Clave

modelos de lenguaje pequeños, mapas conceptuales, Mermaid, evaluación neuro-simbólica, teoría de Novak, educación, generación automática de contenido educativo.

1. Introducción

En el ámbito universitario, los estudiantes enfrentan el constante desafío de asimilar y relacionar grandes volúmenes de información textual. Aunque los resúmenes tradicionales son la herramienta de estudio predominante, su lectura pasiva suele resultar tediosa y, en muchos casos, ineficiente para estructurar un conocimiento interconectado. Frente a esta limitación, la literatura pedagógica demuestra que las representaciones visuales, como los mapas conceptuales, superan ampliamente al texto plano al forzar al alumno a organizar los conceptos de manera jerárquica y relacional. Esto es consistente con la Teoría de la Carga Cognitiva [1], que sostiene que la memoria de trabajo humana tiene una capacidad limitada, y que estructurar la información en artefactos intermedios —como diagramas o esquemas jerárquicos— reduce la carga cognitiva extraña, liberando capacidad para el aprendizaje efectivo. Sin embargo, la construcción manual de estos diagramas demanda una carga cognitiva y un tiempo considerables, lo que dificulta su adopción masiva.

Con el avance de la Inteligencia Artificial generativa, surge la oportunidad de automatizar la creación de este material didáctico. Los Modelos de Lenguaje (LLMs) han demostrado una alta capacidad para transformar texto libre en salidas estructuradas. En este trabajo se distingue entre LLM (Large Language Model), término general para modelos de gran escala de parámetros —típicamente decenas o cientos de miles de millones—, generalmente comerciales y alojados en la nube (p. ej. GPT-4), y SLM (Small Language Model), modelos de escala reducida —en este trabajo, de 8 a 14 mil millones de parámetros— que pueden ejecutarse localmente con recursos de cómputo moderados. Específicamente, el uso de SLMs ejecutados de forma local representa una alternativa ideal para garantizar la privacidad y reducir costos computacionales. Al aplicar técnicas sistemáticas de Prompt Engineering —como las expuestas en el white paper

de Lee Boonstra para Google [2]— es posible condicionar y restringir las respuestas de la IA. No obstante, depender exclusivamente de un SLM para generar la sintaxis completa de un gráfico suele derivar en alucinaciones, pérdida de formato y errores estructurales severos.

Para resolver esta problemática, este trabajo propone una arquitectura de generación híbrida. En lugar de exigirle al modelo de lenguaje que dibuje el diagrama completo, el SLM se utiliza exclusivamente para la extracción semántica de proposiciones lógicas. Posteriormente, un motor programático transforma estos datos en código visual utilizando la herramienta Mermaid. Para asegurar el rigor académico de los mapas generados, el sistema integra una etapa de evaluación híbrida neuro-simbólica que analiza topológica y semánticamente las respuestas del modelo y exige autocorrecciones antes de emitir el gráfico final.

El resto de este trabajo se organiza de la siguiente manera: la Sección 2 describe la metodología empleada y la arquitectura del sistema propuesto; la Sección 3 presenta los Trabajos Relacionados; y la Sección 4 presenta la Conclusión y los Trabajos Futuros.

2. Metodología

2.1. Arquitectura general

Para evitar que el SLM deba generar directamente la sintaxis completa del diagrama, se diseñó una arquitectura de generación en cuatro fases, donde el modelo de lenguaje se emplea exclusivamente para tareas de extracción y síntesis semántica, mientras que la construcción del código Mermaid y su validación quedan a cargo de componentes programáticos.

```
graph TD
```

```
A[Texto] --> B[F1: Resumen]
```

```
B --> C[F2: Conceptos]

C --> D[F3: Grafo]

D --> E[F4: Evaluador]

E -->|Aprobado| F[Mapa final]

E -->|Rechazado| D
```

Figura 1. Pipeline de generación de cuatro fases (código Mermaid).

Cada fase corresponde a una etapa reconocida de la metodología de construcción de mapas conceptuales de Novak y Cañas [3], lo que garantiza que el proceso automatizado preserve los principios pedagógicos originales en lugar de imponer una estructura arbitraria. Esta decisión de diseño responde además al mismo principio de carga cognitiva citado en la Sección 1 [1], aplicado esta vez a la tarea de generación en sí: en lugar de solicitar al SLM que resuelva el problema completo en una única invocación, la descomposición en artefactos intermedios (resumen, lista de conceptos, proposiciones) reduce la complejidad efectiva de cada paso individual, y habilita además la validación y corrección independiente de cada etapa antes de avanzar a la siguiente.

2.2. Fase 1: Resumen de Alta Densidad (Chain of Density)

Dado que los textos de entrada pueden superar ampliamente la ventana de contexto óptima para un SLM de 8-14B de parámetros, la primera fase aplica la técnica Chain of Density (CoD), propuesta originalmente por Adams et al. [4] para la generación de resúmenes con GPT-4. El método consiste en solicitar al modelo tres iteraciones sucesivas de resumen: una primera versión "dispersa en entidades", y dos refinamientos donde se

identifican entidades salientes ausentes y se las incorpora sin aumentar la longitud del texto, mediante fusión de oraciones y eliminación de contenido no informativo. Se utiliza únicamente la tercera iteración como entrada para la fase siguiente, priorizando la densidad semántica sobre la extensión.

2.3. Fase 2: Extracción de Conceptos (Parking Lot)

Siguiendo la técnica homónima descrita por Novak y Cañas [3], esta fase solicita al SLM la extracción de entre 10 y 15 conceptos atómicos (máximo 3 palabras cada uno) a partir del resumen generado en la Fase 1. Se instruye explícitamente al modelo para que el primer elemento de la lista corresponda al concepto raíz, estableciendo una jerarquía inicial que se reutiliza tanto en la Fase 3 como en la Fase 4 (métrica M6).

2.4. Fase 3: Construcción del Grafo (Top-Down)

A partir del concepto raíz y la lista de conceptos secundarios, se solicita al SLM que genere las proposiciones (relaciones sujeto-enlace-objeto) que conectan los conceptos, respetando una estructura jerárquica descendente de lo general a lo específico. El modelo devuelve un arreglo JSON con los campos `source`, `relationship`, `target` e `is_cross_link`, que un componente programático transforma en código Mermaid (graph TD), incluyendo un paso de sanitización de identificadores para garantizar que el diagrama resultante sea sintácticamente válido.

Un aspecto de diseño incorporado tras observar en pruebas preliminares que el modelo tendía a producir estructuras puramente jerárquicas sin síntesis transversal, es la instrucción explícita de generar al menos dos enlaces cruzados (cross-links) que conecten conceptos de ramas distintas del árbol —mecanismo que Novak identifica como evidencia de comprensión profunda del dominio. Si la Fase 4 rechaza el mapa generado, esta fase se reejecuta incorporando la retroalimentación del evaluador como contexto adicional del prompt.

2.5. Fase 4: Evaluador Híbrido Neuro-Simbólico

La validación de calidad del mapa generado constituye el aporte metodológico central de este trabajo. En este trabajo, se entiende por neuro-simbólico a un sistema que combina dos tipos de razonamiento complementarios: uno simbólico, basado en reglas fijas y verificables aplicadas sobre la estructura del grafo (análisis de ciclos, conteos, ratios), y uno neuronal, delegado en el propio SLM para evaluar aspectos semánticos

que no admiten una regla explícita (M7).

Antes de calcular las métricas, el sistema aplica una validación estructural previa de tipo binario que descarta automáticamente (score = 0) cualquier mapa con ciclos, proposiciones con campos vacíos, o inconsistencias que impidan un análisis topológico coherente. Superada esta validación, se calculan diez métricas:

Métrica	Puntuación	Descripción
M1	1 5 %	Atomicidad conceptual: ningún concepto excede las 4 palabras ni tiene estructura de oración completa.
M2	1 0 %	Brevedad de enlaces: conectores relacionales de máximo 5 palabras.
M3	1 0 %	Cobertura del parking lot: % de conceptos extraídos efectivamente incorporados al grafo.
M4	1 0 %	Densidad de interconexión: ratio aristas/nodos, penaliza estructuras lineales.
M5	1 5 %	Enlaces cruzados: cantidad de cross-links, mínimo exigido: dos.
M6	1 0 %	Jerarquía descendente: las relaciones fluyen de lo general a lo específico.
M7	1 0 %	Coherencia con la pregunta de enfoque, evaluada por el propio SLM.
M8	5 %	Ausencia de nodos huérfanos del parking lot.

M	5	Balance de ramas, evita que una
9	%	domine el mapa.
M	5	Complejidad estructural: índice
1	%	compuesto de nodos, aristas y cross-
0		links.

El puntaje final surge de un promedio ponderado de las métricas aplicables, y el mapa se considera aprobado si alcanza un umbral de 75/100. En caso de rechazo, el sistema reintenta la Fase 3 hasta un máximo de dos veces, incorporando las sugerencias de corrección generadas automáticamente.

2.6. Configuración experimental

El pipeline se ejecutó utilizando modelos servidos localmente mediante Ollama. La implementación inicial utilizó Llama 3.1 [5] (8B de parámetros) en ejecución local; posteriormente se migró a Qwen2.5 [6] (14B de parámetros) sobre Google Colab, en función de la disponibilidad de recursos computacionales durante el desarrollo. Los textos de entrada del corpus de prueba están en español.

2.7. Resultados Preliminares

Durante las pruebas iniciales se detectó que el campo is_cross_link no era solicitado al SLM en el prompt de la Fase 3, y se fijaba programáticamente en False para todas las proposiciones, por lo que la métrica M5 puntuaba sistemáticamente 0/100. Se corrigió el prompt para exigir explícitamente al menos dos enlaces cruzados. La Tabla 1 compara los resultados del mismo texto de entrada antes y después de la corrección.

M			D
é		A	e
t	P	n	s
r	e	t	p
i	s	e	u
c	o	s	é
a			s
		1	1
M	1	0	0
l	5	0	0
	%	.	.
		0	0
M	1	1	1

		0	0
	0	0	0
2	%	.	.
		0	0
		9	1
M	1	3	0
3	0	.	0
	%	3	.
			0
		4	5
M	1	6	4
4	0	.	.
	%	4	5
	1	0	6
M	5	.	6
5	%	0	.
			7
		1	1
	1	0	0
M	0	0	0
6	%	.	.
		0	0
		7	6
M	1	0	0
7	0	.	.
	%	0	0
		8	1
M	5	0	0
8	%	.	0
		0	.
			0
		8	8
M	5	0	0
9	%	.	.
		0	0
		6	
M	5	5	1
1	%	.	0
0		0	0
			.

T		7	8
o		0	4
t	—	.	.
a		8	7
l			

Tabla 1. Comparación de puntajes por métrica, mismo texto, antes y después de la corrección.

Con el objetivo de obtener una estimación más representativa del desempeño del pipeline, se construyó un corpus de siete textos de prueba en español (350-600 palabras cada uno), correspondientes a distintas temáticas de Ingeniería en Sistemas de Información. La Tabla 2 resume los resultados de una de las corridas.

T		A
ex	S	p
to	c	r
(te	o	o
m	r	b
a)	e	a
		d
		o
B		
as	8	
es	6	S
de	.	í
da	5	
to		
s		
In		
ter	8	
ru	4	S
pc	.	í
io	2	
ne		
s		
Pr	8	
ue	4	S
ba	.	í
l	9	

R		
ed		
es	8	
ne	6	S
ur	.	í
on	0	
al		
es		
B		
as		
es	8	
de	6	S
da	.	í
to	6	
s		
(2		
)		
Si		
st		
e		
m	0	
as	.	N
op	0	o
er		
ati		
vo		
s		
M		
et		
od		
ol	7	
og	4	N
ía	.	o
s	2	
ág		
ile		
s		

Tabla 2. Resultados del pipeline sobre el corpus de 7 textos.

Un hallazgo no anticipado, pero metodológicamente relevante, surgió al ejecutar el mismo corpus en cuatro oportunidades sucesivas, sin modificar código ni textos de entrada. La Tabla 3 compara los resultados agregados de las cuatro corridas.

I
n
d
i
c
a
d
o
r

S
c
o
r
e

p
r
o
m
e
d
i
o

A
p
r
o
b
a
d
o
s

F
a
l
l
o
s

e
s
t
r
u
c

A

B

C

D

5

7

6

8

1

1

1

5

.

.

.

.

5

8

4

2

1

7

7

1

4

1

1

0

%

%

%

0

%

2

1

2

0

9

4

9

%

%

%

%

t
u
r
a
l
e
s

Tabla 3. Resultados agregados de 4 corridas independientes del mismo corpus.

A través de las 4 corridas, el score promedio por corrida fue de 67.5 ± 14.4 puntos (media $\pm$ desvío estándar muestral; rango 51.5-85.2), y la tasa de aprobación fue de $64.3\% \pm 35.9$ puntos porcentuales (rango 14.3%-100%). La magnitud de esta dispersión confirma que la variabilidad estocástica del SLM es un factor determinante en el desempeño observado del sistema, y no un artefacto de una única corrida atípica.

El desglose por métrica está disponible para las Corridas B y D ($n=6$ y $n=7$ textos válidos respectivamente). En ambas, M4 (Densidad de Interconexión) fue el cuello de botella más consistente: 55.7/100 en B y 53.4/100 en D. Esto es coherente con una tensión de diseño entre M4 y M6: los mapas jerárquicos bien formados naturalmente producen varios nodos hoja con una única conexión, lo cual penaliza el ratio aristas/nodos que usa M4. M1, M2 y M10 obtuvieron 100/100 en ambas corridas, indicando que las restricciones puramente léxicas y la complejidad estructural agregada son las dimensiones más estables.

Respecto a los fallos estructurales duros, dos textos concentraron la mayoría de los casos a lo largo de las 4 corridas: texto_02_bases_de_datos.txt falló en A y C (2/4), y texto_03_sistemas_operativos.txt falló en B y C (2/4).

Adicionalmente, se identificaron tres hallazgos cualitativos: (i) con texto de entrada en español, uno de los mapas generados contuvo etiquetas íntegramente en inglés, atribuible a que los prompts del sistema están escritos en inglés; (ii) se observó al menos un error ortográfico en las etiquetas generadas por el modelo; y (iii) en un caso, un enlace cruzado ubicó incorrectamente dos técnicas de mitigación (RAG y RLHF) como subtipos del problema que buscan mitigar.

3. Trabajos Relacionados

La generación automática de mapas conceptuales mediante Inteligencia

Artificial ha sido objeto de estudio reciente. Un antecedente fundamental es la investigación de Schicchi et al. [7], quienes evaluaron la eficacia de los LLMs para asistir en la creación de estas herramientas de aprendizaje, empleando ChatGPT para procesar textos curriculares y generar la estructura del mapa en formato PlantUML, con resultados comparables a mapas elaborados manualmente por docentes expertos.

A diferencia de ese enfoque, que depende de un modelo comercial de gran escala en la nube, el presente trabajo propone un pipeline ejecutado de forma completamente local utilizando un SLM, compensando su menor capacidad de razonamiento con una fase de evaluación híbrida neuro-simbólica. El diseño se apoya además en las técnicas de "Parking Lot" de Novak y Cañas [3] y de Chain of Density de Adams et al. [4].

4. Conclusión y Trabajos Futuros

Este trabajo presentó un agente para la generación automática de mapas conceptuales en formato Mermaid a partir de texto libre, mediante una arquitectura híbrida de cuatro fases que separa la extracción semántica —a cargo de un SLM ejecutado localmente— de la construcción y validación estructural del grafo, a cargo de componentes programáticos y un evaluador neuro-simbólico.

El hallazgo más relevante de la experimentación es la magnitud de la variabilidad observada entre corridas del mismo corpus: la tasa de aprobación varió entre 14.3% y 100% (media $64.3\% \pm 35.9$ puntos porcentuales), sin ningún cambio de código ni de datos de entrada. Como limitación central, ninguna de las diez métricas del evaluador verifica la corrección factual de las relaciones extraídas.

Como trabajo futuro se identifican: diseño experimental con repetición (evaluar cada texto 3-5 veces); evaluación a escala y comparación con expertos humanos; comparación español/inglés; consistencia de idioma en los prompts; verificación factual contra el texto fuente; recalibración de M4 y M5; estudio de susceptibilidad al fallo por texto; y exploración de aplicación a esquemas de post-entrenamiento teacher/student.

Referencias

- [1] Sweller, J. (1988). Cognitive Load During Problem Solving: Effects on Learning. *Cognitive Science*, 12(2), 257–285.
- [2] Boonstra, L. (2024). Prompt Engineering. Google / Kaggle whitepaper.

- [3] Novak, J. D., & Cañas, A. J. (2008). The Theory Underlying Concept Maps and How to Construct and Use Them. Technical Report IHMC CmapTools 2006-01 Rev 01-2008. Florida Institute for Human and Machine Cognition, Pensacola.
- [4] Adams, G., Fabbri, A., Ladhak, F., Lehman, E., & Elhadad, N. (2023). From Sparse to Dense: GPT-4 Summarization with Chain of Density Prompting. Proceedings of the 4th New Frontiers in Summarization Workshop (NewSum 2023), pp. 68–74. ACL, Singapur.
- [5] Grattafiori, A., Dubey, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783.
- [6] Qwen Team (2024). Qwen2.5 Technical Report. arXiv:2412.15115.
- [7] Schicchi, D., Limongelli, C., Monteleone, V., & Taibi, D. (2026). A closer look at ChatGPT's role in concept map generation for education. Interactive Learning Environments, 34(1), 276–296.